

Backup Monitoring and Verification

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Why Backup Monitoring Matters](#)
 3. [pg_verifybackup](#)
 4. [pg_stat_archiver](#)
 5. [pgBackRest Monitoring](#)
 6. [Testing Restores](#)
 7. [Backup Age Alerting](#)
 8. [WAL Archive Gap Detection](#)
 9. [SQL and Shell Examples](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions and Answers](#)
 14. [Exercises with Solutions](#)
 15. [Cross-references](#)
-

Learning Objectives

After working through this document you will be able to:

- Explain why backup monitoring is as critical as taking backups in the first place
 - Use `pg_verifybackup` to validate a base backup's manifest, checksums, and WAL completeness
 - Query `pg_stat_archiver` to detect WAL archive failures in real time
 - Parse `pgbackrest info --output=json` output to check backup age and WAL archive health
 - Design and implement a monthly restore drill procedure with documented RTO
 - Write SQL and shell alerting rules for backup age, archive failures, and WAL gaps
 - Identify the most common monitoring failure modes and explain how to prevent them
-

Theory: Why Backup Monitoring Matters

Silent Failures Kill RTO Guarantees

Taking a backup is not enough. A backup that has never been verified is a hypothesis, not a guarantee. Silent failures are the most dangerous category in backup operations:

- **Archive command succeeds locally but the file is corrupt on the remote storage.** The `archive_command` returned exit code 0 because the `cp` or `aws s3 cp` call succeeded at the OS level, but the file written to S3 was truncated due to a network interruption. Nobody knows until the restore attempt fails at 3 AM during an incident.
- **pg_stat_archiver shows increasing failed_count that nobody is monitoring.** WAL segments are piling up in `pg_wal/archive_status/*.ready` while the DBA team's attention is elsewhere. Disk fills up. Database crashes.
- **The last successful backup is 72 hours old because the backup job silently started failing.** The monitoring alert was set on the backup job's exit code, not on the timestamp of the last verified

backup. The job fails immediately with a connection error and nobody notices because the alert was misconfigured.

- **A restore has never been tested.** Backups are taken faithfully every night. The `pg_hba.conf` on the backup server prevents the restore script from connecting. The `pg_wal` directory is on a separate mount that was not included in the backup. The PITR target timestamp was specified in the wrong timezone. These failures surface only during a real incident.

The operational contract is: **a backup is only as good as its last verified restore**. Monitoring closes the loop between "backup was taken" and "backup can be restored within RTO."

pg_verifybackup

Overview

`pg_verifybackup` (introduced in PostgreSQL 13) verifies a base backup taken with `pg_basebackup` by checking the backup manifest against the actual files on disk. It validates file presence, sizes, and checksums, and optionally verifies that the WAL required to bring the backup to a consistent state is present.

Basic Usage

```
# Verify a backup at the default manifest path
pg_verifybackup /var/lib/postgresql/backups/base_2026-06-06

# Verify with explicit manifest path
pg_verifybackup --manifest-path=/backups/manifests/backup_manifest_2026-06-06 \
    /var/lib/postgresql/backups/base_2026-06-06

# Skip checksum verification (faster, checks presence and sizes only)
pg_verifybackup --no-checksums /var/lib/postgresql/backups/base_2026-06-06

# Skip WAL verification (when WAL is stored separately)
pg_verifybackup --no-verify-checksums --ignore=pg_wal \
    /var/lib/postgresql/backups/base_2026-06-06
```

What pg_verifybackup Checks

1. **Manifest existence and integrity** — Reads `backup_manifest` from the backup directory. The manifest itself contains a SHA256 checksum of its own contents to detect manifest corruption.
2. **File presence** — Every file listed in the manifest must be present in the backup directory. Missing files cause immediate verification failure.
3. **File sizes** — Actual file sizes are compared against sizes recorded in the manifest. A truncated file fails this check.
4. **File checksums** — For each file in the manifest, `pg_verifybackup` computes the listed algorithm (SHA256 by default) and compares against the recorded hash. This detects silent data corruption in backup storage.
5. **WAL completeness** — Checks that the WAL segment range needed to bring the backup to a consistent state (from the backup's start LSN to end LSN) is present in the `pg_wal` subdirectory of

the backup or in the path specified by `--wal-directory` .

Important Flags

Flag	Effect
<code>--no-checksums</code>	Skip per-file checksum computation (faster, weaker)
<code>--manifest-path=PATH</code>	Specify manifest location if not in backup root
<code>--wal-directory=DIR</code>	Look for WAL in a separate directory
<code>--ignore=PATTERN</code>	Skip verification of files matching pattern
<code>--progress</code>	Show a progress indicator during checksum computation
<code>--quiet</code>	Suppress informational messages, only report errors

Exit Codes

- **0** — Verification passed, backup is intact
- **Non-zero** — Verification failed; the error message identifies the specific failure

Automation Example

```
#!/bin/bash
# verify_latest_backup.sh
BACKUP_DIR="/backups/pg/$(ls -t /backups/pg/ | head -1)"
if pg_verifybackup --quiet "$BACKUP_DIR"; then
    echo "BACKUP_VERIFIED OK: $BACKUP_DIR"
else
    echo "BACKUP_VERIFY_FAILED: $BACKUP_DIR" | mail -s "ALERT: Backup Verification Failed" dba-team@company.com
    exit 1
fi
```

pg_stat_archiver

Overview

`pg_stat_archiver` is a single-row system view that tracks the activity of PostgreSQL's WAL archiver process. It is the primary real-time indicator of WAL archive health and must be part of every production monitoring setup.

Columns

Column	Type	Description
<code>archived_count</code>	<code>bigint</code>	Total WAL files successfully archived since last stats reset
<code>last_archived_wal</code>	<code>text</code>	Name of the last successfully archived WAL segment
<code>last_archived_time</code>	<code>timestampz</code>	Timestamp of the last successful archive

failed_count	bigint	Total archive attempts that failed since last stats reset
last_failed_wal	text	Name of the WAL segment that last failed to archive
last_failed_time	timestampz	Timestamp of the last archive failure
stats_reset	timestampz	When statistics were last reset

Query to Detect Archive Failures

```
-- Check archiver health: should run in monitoring every 1-5 minutes
SELECT
    archived_count,
    last_archived_wal,
    last_archived_time,
    EXTRACT(EPOCH FROM (now() - last_archived_time)) / 60 AS
minutes_since_last_archive,
    failed_count,
    last_failed_wal,
    last_failed_time,
    CASE
        WHEN failed_count > 0 AND last_failed_time > last_archived_time
            THEN 'ALERT: Archive is currently failing'
        WHEN now() - last_archived_time > INTERVAL '30 minutes'
            THEN 'WARNING: No successful archive in 30+ minutes'
        ELSE 'OK'
    END AS archive_status
FROM pg_stat_archiver;
```

Alert: failed_count Increasing = WAL Archive Broken

The most important signal in `pg_stat_archiver` is a monotonically increasing `failed_count`. A single failure may be transient (network blip, temporary S3 throttle), but if `failed_count` is higher on each polling cycle, the WAL archive pipeline is broken:

- The `archive_command` is returning non-zero exit codes
- WAL segment files are accumulating in `pg_wal/archive_status/` with `.ready` suffix
- If `pg_wal` fills the disk, the primary will pause all writes and potentially crash

```
-- Compare failed_count across two polling intervals
-- Store previous value in your monitoring system and alert on delta > 0
SELECT failed_count, last_failed_wal, last_failed_time
FROM pg_stat_archiver
WHERE failed_count > 0;
```

```
# Shell one-liner to check archive health
psql -tAc "SELECT CASE WHEN failed_count > 0 AND last_failed_time >
last_archived_time \
    THEN 'FAILING: ' || last_failed_wal ELSE 'OK' END FROM pg_stat_archiver;"
```

pgBackRest Monitoring

Overview

pgBackRest is a comprehensive backup tool for PostgreSQL that provides incremental/differential backups, WAL archiving, parallel backup/restore, and built-in backup verification. Its `info` command is the primary monitoring interface.

pgbackrest info Command

```
# Human-readable output
pgbackrest --stanza=main info

# JSON output for programmatic parsing
pgbackrest --stanza=main info --output=json

# JSON for a specific repository
pgbackrest --stanza=main --repo=1 info --output=json
```

Parsing JSON Output

The JSON output from `pgbackrest info` includes backup metadata, WAL archive status, and repository health:

```
# Check last backup time (requires jq)
pgbackrest --stanza=main info --output=json | \
jq -r '.[0].backup[-1].timestamp.stop | . as $ts | ($ts | todate)'
```

```
# Get last backup age in hours
pgbackrest --stanza=main info --output=json | \
jq -r '.[0].backup[-1].timestamp.stop' | \
xargs -I{} bash -c 'echo $(( ($(date +%s) - {}) / 3600 )) hours ago'
```

```
# Check WAL archive status
pgbackrest --stanza=main info --output=json | \
jq -r '.[0].archive[0] | "min: \(.min) max: \(.max) database: \(.database)''
```

```
# Check backup size
pgbackrest --stanza=main info --output=json | \
jq -r '.[0].backup[-1] | "size: \(.info.size) delta: \(.info.delta)''
```

Shell One-Liner to Check Backup Age

```
#!/bin/bash
# Returns exit 1 if last backup is older than MAX_AGE_HOURS
MAX_AGE_HOURS=25
STANZA="main"

LAST_BACKUP_EPOCH=$(pgbackrest --stanza="$STANZA" info --output=json | \
```

```

jq -r '[0].backup[-1].timestamp.stop')

if [ -z "$LAST_BACKUP_EPOCH" ] || [ "$LAST_BACKUP_EPOCH" = "null" ]; then
    echo "CRITICAL: No backups found for stanza $STANZA"
    exit 2
fi

NOW=$(date +%s)
AGE_HOURS=$(( (NOW - LAST_BACKUP_EPOCH) / 3600 ))

if [ "$AGE_HOURS" -gt "$MAX_AGE_HOURS" ]; then
    echo "CRITICAL: Last backup is ${AGE_HOURS}h old (threshold: ${MAX_AGE_HOURS}h)"
    exit 1
else
    echo "OK: Last backup is ${AGE_HOURS}h old"
    exit 0
fi

```

pgbackrest check Command

```

# Verify configuration and archive pipeline end-to-end
pgbackrest --stanza=main check

# Verbose output showing each verification step
pgbackrest --stanza=main --log-level-console=detail check

```

The `check` command switches a WAL segment, archives it, and verifies the archived segment is retrievable — confirming the full archive pipeline works.

Testing Restores

Why Monthly Restore Drills Are Non-Negotiable

A backup that has never been restored is a liability, not an asset. Common failure modes only discovered during a restore drill:

- `pg_hba.conf` on the restore server does not permit the application user
- `postgresql.conf` memory settings from the source are inappropriate for the restore server
- PITR timestamp was specified in UTC but application logs use local time
- WAL is present but the timeline does not match (split-brain after failed failover)
- The restore script has a bug that has existed for 8 months

Monthly Restore Drill Procedure (Step by Step)

Prerequisites: A separate server or container with the same PostgreSQL version, access to backup storage, documented RTO target.

Step 1: Prepare the restore environment

```

# Stop any existing PostgreSQL instance on the restore server
sudo systemctl stop postgresql

```

```
# Clear the data directory
sudo -u postgres rm -rf /var/lib/postgresql/14/main/*
```

Step 2: Restore the base backup

```
# Using pgBackRest
sudo -u postgres pgbackrest --stanza=main --delta restore

# Using pg_basebackup (if taking fresh backup for drill)
sudo -u postgres pg_basebackup \
  -h primary_host -U replication_user \
  -D /var/lib/postgresql/14/main \
  -Fp -Xs -P -R
```

Step 3: Configure recovery target (if testing PITR)

```
# Edit postgresql.conf or recovery.conf (PG11 and earlier)
# In PG12+, use postgresql.conf:
cat >> /var/lib/postgresql/14/main/postgresql.conf <<EOF
restore_command = 'pgbackrest --stanza=main archive-get %f %p'
recovery_target_time = '2026-06-06 02:00:00 UTC'
recovery_target_action = 'promote'
EOF

# Create recovery signal file
touch /var/lib/postgresql/14/main/recovery.signal
```

Step 4: Start PostgreSQL and monitor recovery

```
sudo systemctl start postgresql
# Follow recovery progress
sudo tail -f /var/log/postgresql/postgresql-14-main.log | \
  grep -E '(recovery|checkpoint|consistent|promoted)'
```

Step 5: Verify data integrity after restore

```
-- Check database list
\l

-- Count rows in key tables (compare against known-good baselines)
SELECT schemaname, tablename, n_live_tup
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC
LIMIT 20;

-- Check for corruption using pg_catalog
SELECT count(*) FROM pg_class;
```

```
-- Run application-level consistency checks
-- (table row counts, referential integrity spot checks, recent transaction data)
```

Step 6: Document restore time (actual RTO)

```
# Record timestamps
RESTORE_START=$(date -d "when you started step 1" +%s)
RESTORE_END=$(date +%s)
ACTUAL_RTO=$(( (RESTORE_END - RESTORE_START) / 60 ))
echo "Actual RTO: ${ACTUAL_RTO} minutes (target: $TARGET_RTO_MINUTES minutes)"
```

Step 7: Document findings and update runbooks Record: actual RTO, any deviations from the runbook, WAL availability confirmation, and whether the restore met the RTO target.

Backup Age Alerting

SQL Alert: Last Backup Older Than 25 Hours

```
-- Run via monitoring system (Nagios, Prometheus postgres_exporter custom query, etc.)
-- Assumes a backup_log table populated by your backup script
SELECT
    CASE
        WHEN max(completed_at) < now() - INTERVAL '25 hours'
        THEN 'CRITICAL: Last backup ' ||
            EXTRACT(EPOCH FROM (now() - max(completed_at)))/3600 ||
            ' hours ago'
        WHEN max(completed_at) < now() - INTERVAL '13 hours'
        THEN 'WARNING: Last backup ' ||
            EXTRACT(EPOCH FROM (now() - max(completed_at)))/3600 ||
            ' hours ago'
        ELSE 'OK: Last backup ' ||
            EXTRACT(EPOCH FROM (now() - max(completed_at)))/60 ||
            ' minutes ago'
        END AS backup_age_status
FROM backup_log
WHERE status = 'success';
```

Shell Alert Using pgBackRest JSON

```
#!/bin/bash
# nagios-style check: exits 0=OK, 1=WARNING, 2=CRITICAL
WARN_HOURS=13
CRIT_HOURS=25
STANZA=${1:-main}

LAST_STOP=$(pgbackrest --stanza="$STANZA" info --output=json 2>/dev/null | \
jq -r '.[0].backup[-1].timestamp.stop // empty')
```

```
[ -z "$LAST_STOP" ] && { echo "UNKNOWN: No backup data"; exit 3; }

NOW=$(date +%s)
AGE_H=$(( (NOW - LAST_STOP) / 3600 ))

if [ "$AGE_H" -ge "$CRIT_HOURS" ]; then echo "CRITICAL: backup age ${AGE_H}h";
exit 2
elif [ "$AGE_H" -ge "$WARN_HOURS" ]; then echo "WARNING: backup age ${AGE_H}h"; exit
1
else echo "OK: backup age ${AGE_H}h"; exit 0
fi
```

Prometheus / postgres_exporter Custom Query

```
# /etc/postgres_exporter/queries.yaml
pg_backup_age:
  query: |
    SELECT
      EXTRACT(EPOCH FROM (now() - max(completed_at))) AS seconds_since_last_backup
    FROM backup_log
    WHERE status = 'success'
  metrics:
    - seconds_since_last_backup:
        usage: GAUGE
        description: Seconds since the last successful backup completed
```

```
# Prometheus alert rule
- alert: PostgreSQLBackupTooOld
  expr: pg_backup_age_seconds_since_last_backup > 90000 # 25 hours
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "PostgreSQL backup older than 25 hours on {{ $labels.instance }}"
```

WAL Archive Gap Detection

pg_stat_archiver failed_count Monitoring

The `failed_count` column in `pg_stat_archiver` is a cumulative counter that only increases. A monitoring system should record the value on each poll interval and alert when the delta is non-zero:

```
-- Snapshot-based monitoring query
SELECT
  failed_count,
  last_failed_wal,
  last_failed_time,
  archived_count,
  last_archived_time,
```

```
-- Alert if failures are more recent than last success
(last_failed_time > last_archived_time) AS archive_currently_failing
FROM pg_stat_archiver;
```

Using pg_ls_archive_statusdir()

PostgreSQL 10+ provides `pg_ls_archive_statusdir()` to list files in `pg_wal/archive_status/` :

```
-- Files awaiting archiving (.ready) or already archived (.done)
SELECT name, size, modification
FROM pg_ls_archive_statusdir()
ORDER BY modification DESC;

-- Count of pending (not yet archived) WAL segments
SELECT count(*) AS pending_wal_segments
FROM pg_ls_archive_statusdir()
WHERE name LIKE '%.ready';

-- Alert if more than 10 segments are waiting
SELECT
    count(*) AS pending_segments,
    CASE WHEN count(*) > 10 THEN 'ALERT: WAL archive backlog' ELSE 'OK' END AS
status
FROM pg_ls_archive_statusdir()
WHERE name LIKE '%.ready';
```

Checking for Gaps in the Archive

```
#!/bin/bash
# Check for gaps between current WAL position and last archived segment
CURRENT_WAL=$(psql -tAc "SELECT pg_walfile_name(pg_current_wal_lsn());")
LAST_ARCHIVED=$(psql -tAc "SELECT last_archived_wal FROM pg_stat_archiver;")

echo "Current WAL: $CURRENT_WAL"
echo "Last archived: $LAST_ARCHIVED"

# Use pgbackrest to verify archive integrity
pgbackrest --stanza=main archive-get --log-level-console=info \
"$LAST_ARCHIVED" /tmp/wal_verify_test && \
echo "Archive retrievable OK" || echo "ALERT: Archive retrieve failed"
```

SQL and Shell Examples

```
-- 1. Check archiver status with human-readable age
SELECT
    last_archived_wal,
    to_char(last_archived_time, 'YYYY-MM-DD HH24:MI:SS') AS last_archived,
    age(now(), last_archived_time) AS time_since_archive,
```

```

        failed_count,
        last_failed_wal
FROM pg_stat_archiver;

-- 2. Count pending WAL files awaiting archive
SELECT count(*) AS ready_to_archive
FROM pg_ls_archive_statusdir()
WHERE name LIKE '%.ready';

-- 3. Identify the oldest unarchived WAL segment
SELECT name AS oldest_pending
FROM pg_ls_archive_statusdir()
WHERE name LIKE '%.ready'
ORDER BY name
LIMIT 1;

-- 4. Check whether archive is falling behind current WAL
SELECT
    pg_walfile_name(pg_current_wal_lsn()) AS current_wal,
    (SELECT last_archived_wal FROM pg_stat_archiver) AS last_archived,
    pg_walfile_name(pg_current_wal_lsn()) > (SELECT last_archived_wal FROM
pg_stat_archiver)
    AS archive_lag_exists;

-- 5. Backup age check via a backup_log tracking table
SELECT
    backup_type,
    completed_at,
    backup_size_bytes,
    duration_seconds,
    EXTRACT(EPOCH FROM (now() - completed_at)) / 3600 AS age_hours
FROM backup_log
WHERE status = 'success'
ORDER BY completed_at DESC
LIMIT 5;

-- 6. Alert query: critical if no successful backup in 25 hours
SELECT
    CASE
        WHEN max(completed_at) IS NULL THEN 'CRITICAL: no backups recorded'
        WHEN max(completed_at) < now() - INTERVAL '25 hours' THEN 'CRITICAL'
        WHEN max(completed_at) < now() - INTERVAL '13 hours' THEN 'WARNING'
        ELSE 'OK'
    END AS status,
    max(completed_at) AS last_backup_time
FROM backup_log
WHERE status = 'success';

-- 7. List all pg_toast tables and their sizes (to monitor TOAST bloat in critical
tables)
SELECT
    n.nspname AS schema,

```

```

        c.relname AS toast_table,
        t.relname AS parent_table,
        pg_size_pretty(pg_relation_size(c.oid)) AS toast_size
FROM pg_class c
JOIN pg_class t ON t.reltoastrelid = c.oid
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 't'
ORDER BY pg_relation_size(c.oid) DESC;

-- 8. Check relfrozenxid age for all tables (wraparound risk)
SELECT
    schemaname,
    tablename,
    age(relfrozenxid) AS xid_age,
    CASE
        WHEN age(relfrozenxid) > 1500000000 THEN 'CRITICAL: near wraparound'
        WHEN age(relfrozenxid) > 750000000 THEN 'WARNING'
        ELSE 'OK'
    END AS freeze_status
FROM pg_stat_user_tables
JOIN pg_class ON relname = tablename
ORDER BY age(relfrozenxid) DESC
LIMIT 20;

-- 9. WAL generation rate (bytes per second)
SELECT
    wal_bytes / extract(epoch from stats_age) AS wal_bytes_per_sec,
    pg_size_pretty(wal_bytes / extract(epoch from stats_age)::bigint) || '/s' AS
wal_rate
FROM (
    SELECT wal_bytes, now() - stats_reset AS stats_age
    FROM pg_stat_wal
) w;

-- 10. Check pg_stat_replication for streaming replica WAL lag
SELECT
    client_addr,
    application_name,
    state,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag_bytes,
    replay_lag
FROM pg_stat_replication;

-- 11. Check datfrozenxid for all databases
SELECT
    datname,
    age(datfrozenxid) AS frozen_xid_age,
    current_setting('autovacuum_freeze_max_age')::int - age(datfrozenxid) AS
xids_until_forced_vacuum

```

```

FROM pg_database
ORDER BY age(datfrozenxid) DESC;

-- 12. Find tables with high dead tuple ratio
SELECT
    schemaname,
    tablename,
    n_live_tup,
    n_dead_tup,
    round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
    last_autovacuum,
    last_vacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY dead_pct DESC
LIMIT 20;

-- 13. Verify pg_basebackup can connect (pre-backup connectivity check)
-- Run as superuser or replication role
SELECT pg_is_in_backup(), pg_backup_start_time();

-- 14. Check if WAL archiving is enabled
SELECT name, setting
FROM pg_settings
WHERE name IN ('archive_mode', 'archive_command', 'wal_level');

-- 15. Identify replication slots that might block VACUUM
SELECT
    slot_name,
    plugin,
    slot_type,
    database,
    active,
    xmin,
    catalog_xmin,
    restart_lsn,
    age(xmin) AS xmin_age
FROM pg_replication_slots
ORDER BY age(xmin) DESC;

```

```

# Shell example: pgBackRest backup status one-liner
pgbackrest --stanza=main info --output=json | \
    jq -r '.[0].backup[] | [.type, (.timestamp.stop | todate), .info.size] | @tsv'

# Shell example: alert if any .ready files are older than 1 hour
find /var/lib/postgresql/14/main/pg_wal/archive_status/ \
    -name "*.ready" -mmin +60 \
    -exec echo "ALERT: old unarchived WAL: {}" \;

```

Common Mistakes

1. **Monitoring backup job exit codes instead of backup age.** A backup job that exits 0 but produces an empty or corrupt backup file will never alert. Always monitor the timestamp of the last verified, complete backup, not the job exit code.
 2. **Never testing restores.** A backup procedure that has not produced a successful restore in the past 30 days is an unknown. Always run monthly restore drills to a separate server and document the actual time to recovery.
 3. **Ignoring `pg_stat_archiver.failed_count`.** This counter increases silently while WAL accumulates in `pg_wal`. When the disk fills, the primary stops and an emergency incident begins. Add a monitoring check that alerts on any increase in `failed_count`.
 4. **Assuming `archive_command` succeeding means the archive is usable.** The `archive_command` exit code only confirms the shell command ran without error. Always use `pgbackrest check` or `pg_verifybackup` to confirm end-to-end archive integrity.
 5. **Not accounting for timezone in PITR recovery targets.** `recovery_target_time` in `recovery.conf` or `postgresql.conf` must be specified in a format that matches PostgreSQL's `timezone` setting. A mismatch causes the restore to target the wrong point in time, leading to data loss or an inconsistent database state.
-

Best Practices

1. **Run `pg_verifybackup` on every base backup before considering it valid.** Integrate it into the backup job pipeline: take backup → verify backup → record result in monitoring. Treat a failed verification as a failed backup.
 2. **Monitor `pg_stat_archiver` every 5 minutes at minimum.** Alert on `failed_count > 0 AND last_failed_time > last_archived_time` as a critical signal. Alert on `now() - last_archived_time > 30 minutes` as a warning.
 3. **Run a restore drill monthly with documented RTO.** The drill must include: restore to a separate server, verify data integrity, and document the actual time from start to database-serving-queries. Compare against RTO target and escalate if exceeded.
 4. **Store backups in at least two locations.** Local backup + remote object storage (S3/GCS). A single-location backup fails to protect against storage failures, ransomware, and accidental deletion of the entire backup host.
 5. **Use structured backup logs in a database table.** Record backup type, start time, end time, size, verification result, and WAL range covered. This enables SQL-based monitoring queries, historical trending, and audit trails for compliance.
-

Performance Considerations

- `pg_verifybackup` with checksum verification is CPU-bound and can take significant time for large backups. For a 1 TB backup with SHA256 checksums, expect 30–60 minutes depending on disk speed. Schedule verification to run off-peak or in parallel with backup upload.

- The `archive_command` is called synchronously by the WAL archiver for each 16 MB segment. If the command is slow (e.g., S3 upload with high latency), WAL segments will queue up in `pg_wal`. Keep `archive_command` fast or use asynchronous archiving tools (pgBackRest with async archiving, WAL-G).
- pgBackRest `--process-max` controls parallelism for backup and restore. For large databases, increasing this value significantly reduces backup and restore time. A 4 TB database that takes 4 hours with `--process-max=1` may take 45 minutes with `--process-max=8` on fast storage.
- Restore drills using `--delta` mode (pgBackRest) are much faster than full restores when only a subset of files changed since the last drill. Use `--delta` for monthly drills on large databases to keep the drill time within the maintenance window.

Interview Questions and Answers

Q1: What is the difference between `pg_verifybackup` and a test restore?

`pg_verifybackup` verifies that the backup files on disk match the manifest (correct files, correct sizes, correct checksums) and that the required WAL is present. It confirms backup integrity without actually running PostgreSQL. A test restore goes further: it starts a PostgreSQL instance from the backup, applies WAL, verifies the database reaches a consistent state, and confirms that application-level data is present and correct. Both are necessary — `pg_verifybackup` is a fast automated check, while a test restore is the only way to confirm the full recovery process works end to end.

Q2: How would you detect that WAL archiving has silently stopped working?

Query `pg_stat_archiver` and check two conditions: (a) `failed_count` has increased since the last poll — this means the archive command is actively failing; (b) `now() - last_archived_time > threshold` — this means no new WAL has been archived in too long, which could indicate archiving is paused or the primary has been idle. Additionally, query `pg_ls_archive_statusdir()` and count files with `.ready` suffix — a growing count means segments are queuing up unarchived. Alert on both conditions.

Q3: A developer reports that a PITR restore recovered the database to the wrong point in time. What are the most likely causes?

The most common causes: (1) timezone mismatch — `recovery_target_time` was specified without a timezone suffix and PostgreSQL interpreted it in a different timezone than intended; (2) the WAL for the target timestamp was not present in the archive (gap in WAL coverage); (3) `recovery_target_inclusive` was set incorrectly, including or excluding the boundary transaction; (4) the backup's start LSN was after the target timestamp (cannot recover to a point before the backup was taken). Always specify recovery targets with an explicit timezone offset.

Q4: How do replication slots impact backup and recovery procedures?

Active replication slots prevent WAL segment deletion in `pg_wal` — PostgreSQL retains WAL from the slot's `restart_lsn` forward even if it has been archived. An abandoned slot (e.g., a dropped logical replication subscriber that was not cleaned up) can cause `pg_wal` to fill the disk. Monitor `pg_replication_slots` for inactive slots with growing `restart_lsn` age. During disaster recovery, ensure replication slots on the new primary are recreated or that replicas can reconnect within the WAL retention window.

Q5: What monitoring would you implement to ensure your backup system meets a 1-hour RTO?

RTO of 1 hour requires: (1) backup freshness — last backup must be recent enough that WAL replay from backup end to recovery target fits in the time budget; (2) WAL archive health — `pg_stat_archiver.failed_count` must be zero, and archive lag must be minimal; (3) restore time validation — monthly drills confirm that the actual restore time (base restore + WAL replay) stays under 45 minutes to leave buffer; (4) runbook currency — restore procedure documented and tested by at least two team members; (5) monitoring gaps — alerts for backup age >13h (warning) and >25h (critical), WAL archive failures, and `pg_wal` disk usage >70%.

Q6: How does pgBackRest's incremental backup work and what does it mean for restore speed?

pgBackRest incremental backups copy only files that have changed since the previous backup (full or differential). It uses a manifest comparison between the current file state and the recorded state in the previous backup, copying modified files and hardlinking or referencing unchanged ones. For restore, pgBackRest assembles the full set of required files from multiple backup sets (the latest incremental plus all parent differentials/fulls in the chain). Restore speed for an incremental restore is similar to a full restore in terms of data transferred — you still need all the data files — but backup creation time and storage are much lower. If any backup in the chain is corrupt, the entire chain's restore is at risk, making verification of each backup in the chain critical.

Q7: What does pg_stat_archiver.stats_reset tell you and when would you reset it?

`stats_reset` shows when archiver statistics were last reset, either via `pg_stat_reset_shared('archiver')` or a server restart. A recent `stats_reset` means historical failure counts are lost. You would reset it after resolving an archiving incident to start fresh failure monitoring, but be aware that resetting clears `failed_count` history. In production, avoid resetting archiver stats routinely — the historical failure count can be useful for trend analysis and incident post-mortems. A monitoring system should record the `stats_reset` timestamp and adjust delta calculations when it changes.

Q8: How would you set up end-to-end backup monitoring in a new PostgreSQL environment from scratch?

Starting from scratch: (1) Create a `backup_log` table to record each backup event (type, start/end time, size, WAL range, verification result); (2) configure `archive_command` to a reliable destination and set up `pg_stat_archiver` alerts in the monitoring system; (3) run `pg_verifybackup` after each base backup and log the result; (4) add a Prometheus postgres_exporter custom query for backup age in seconds, alerting at 13h warning and 25h critical; (5) schedule a monthly restore drill with a runbook and RTO documentation requirement; (6) add monitoring for `pg_replication_slots` abandoned slots and `pg_wal` directory size; (7) test the entire monitoring stack by intentionally breaking `archive_command` and verifying alerts fire within the expected time.

Exercises with Solutions

Exercise 1: Write a SQL query to alert if the last successful backup is older than 25 hours, assuming a `backup_log(id, status, completed_at, backup_type)` table exists.

Solution:

```
SELECT
  CASE
    WHEN max(completed_at) < now() - INTERVAL '25 hours'
    THEN 'CRITICAL: backup overdue by ' ||
         round(extract(epoch from (now() - max(completed_at)))/3600, 1) || '
```

```

hours'
    ELSE 'OK'
END AS alert
FROM backup_log
WHERE status = 'success';

```

Exercise 2: Write a shell script that runs `pg_verifybackup` on the most recently created backup directory under `/backups/pg/` and sends an email if verification fails.

Solution:

```

#!/bin/bash
BACKUP_DIR="/backups/pg/$(ls -t /backups/pg/ | head -1)"
LOG="/tmp/pgverify_$(date +%Y%m%d).log"

if pg_verifybackup "$BACKUP_DIR" > "$LOG" 2>&1; then
    echo "Verification OK: $BACKUP_DIR"
else
    mail -s "CRITICAL: pg_verifybackup FAILED for $BACKUP_DIR" \
        dba@company.com < "$LOG"
    exit 1
fi

```

Exercise 3: Using `pg_stat_archiver` and `pg_ls_archive_statusdir()`, write a query that returns 'CRITICAL' if either archive failures are occurring or more than 5 WAL segments are pending archival.

Solution:

```

SELECT
    CASE
        WHEN a.failed_count > 0 AND a.last_failed_time > a.last_archived_time
            THEN 'CRITICAL: archiver is actively failing'
        WHEN p.pending_count > 5
            THEN 'CRITICAL: ' || p.pending_count || ' WAL segments pending archival'
        ELSE 'OK'
    END AS status
FROM pg_stat_archiver a
CROSS JOIN (
    SELECT count(*) AS pending_count
    FROM pg_ls_archive_statusdir()
    WHERE name LIKE '%.ready'
) p;

```

Cross-references

- **09_Backup_Basics/** — `pg_dump`, `pg_basebackup` fundamentals; covers taking backups, this document covers monitoring them
- **10_PITR_Recovery/** — Point-in-time recovery procedure using the WAL archives monitored here
- **13_Replication_HA/** — Streaming replication setup; WAL archiving monitored here also serves as the replication fallback

- **12_Production_PostgreSQL/** — Production monitoring dashboards; backup age and archive health metrics belong on the production ops dashboard
- **PostgreSQL documentation:** `pg_verifybackup` man page, `pg_stat_archiver` reference, `archive_command` configuration
- **pgBackRest documentation:** `info` command JSON schema, `check` command reference, monitoring integration guide